BANCO DE DADOS

com Microsoft SQL Server

AULA 11

JOINs Avançados

LEFT JOIN · RIGHT JOIN · FULL JOIN · SELF JOIN

- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- SELF JOIN
- INNER vs LEFT
- Registros órfãos
- Hierarquias
- Casos práticos

🎯 Objetivos da Aula 11

- ⬅️ Dominar o LEFT JOIN: incluir todos os registros da tabela esquerda
- 🆚 Distinguir quando usar INNER vs LEFT JOIN na prática
- ➡️ Entender o RIGHT JOIN e sua equivalência ao LEFT JOIN
- 🔍 Encontrar registros 'órfãos' (sem correspondência) com IS NULL
- ↔️ Usar FULL JOIN para incluir todos os registros de ambas as tabelas
- 🌳 Modelar hierarquias com SELF JOIN (funcionário → gerente)
- 🔄 Escrever SELF JOIN: uma tabela se unindo a ela mesma
- 💻 Resolver casos reais: clientes sem pedido, produtos sem venda

🔄 Revisão — Aula 10: INNER JOIN

❓ **O que é produto cartesiano e como evitá-lo?**	✅ Combinação de todas as linhas de A × B. Evitar com condição ON correta. N tabelas = N-1 ONs
❓ **O que INNER JOIN retorna?**	✅ Apenas linhas com correspondência em AMBAS as tabelas. Sem par = excluído
❓ **Por que usar alias de tabela?**	✅ Torna a query legível e resolve ambiguidade quando colunas têm o mesmo nome em tabelas diferentes
❓ **Por que COUNT(DISTINCT id) em JOIN 1:N?**	✅ Evita contar o mesmo registro N vezes quando ele aparece em múltiplas linhas do JOIN
❓ **Com 5 tabelas no JOIN, quantas condições ON?**	✅ 4 condições ON (N tabelas = N-1 condições). Verifique sempre!

⚠️ O Problema Silencioso do INNER JOIN

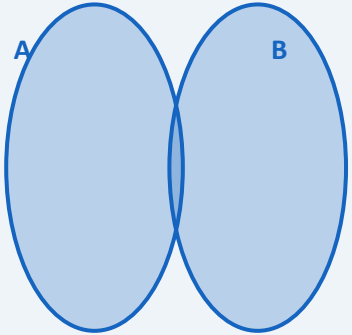
INNER JOIN exclui silenciosamente registros sem correspondência. Em um relatório de clientes, clientes que nunca compraram simplesmente desaparecem — sem aviso algum. Isso pode levar a análises incorretas.

CLIENTES (8 cadastrados)	PEDIDOS (quem comprou?)	INNER JOIN (resultado)
Ana Silva	Ana → pedido 1	Ana ✅
Bruno Santos	Bruno → pedido 2	Bruno ✅
Carla Oliveira	Carla → pedido 3	Carla ✅
Diego Ferreira	Ana → pedido 4	Ana ✅
Eva Costa	Eva → pedido 5	Eva ✅
Felipe Lima	Felipe → pedido 6	Felipe ✅
Gabriela Rocha	—	❌ SUMIU!
Henrique Dias	—	❌ SUMIU!

💡 **Solução: LEFT JOIN inclui TODOS os clientes, mesmo os sem pedido. Clientes sem pedido aparecem com NULL nos campos do pedido.**

Os 4 Tipos de JOIN — Diagramas de Venn

INNER JOIN



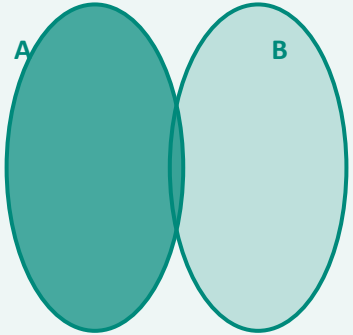
`JOIN / INNER JOIN`

Apenas correspondência

💼

Pedidos COM cliente

LEFT JOIN



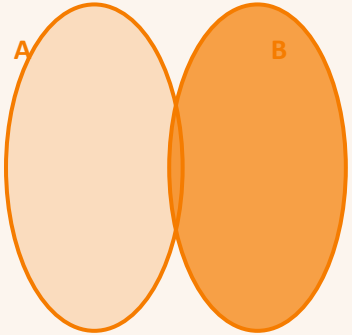
`LEFT JOIN / LEFT OUTER JOIN`

Todos de A + correspondência

💼

TODOS clientes (± pedidos)

RIGHT JOIN



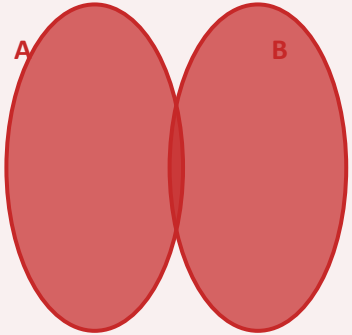
`RIGHT JOIN / RIGHT OUTER JOIN`

Todos de B + correspondência

💼

TODOS pedidos (± cliente)

FULL JOIN



`FULL OUTER JOIN`

Todos de A e B

💼

TUDO: clientes e pedidos

💻 LEFT JOIN — Incluindo Todos da Tabela Esquerda

```sql
-- LEFT JOIN: retorna TODOS os registros da tabela à ESQUERDA
-- Registros sem par na direita aparecem com NULL

-- TODOS os clientes (com ou sem pedido)
SELECT c.nome          AS cliente,
       c.estado,
       p.id_pedido,
       p.status,
       p.total
FROM   CLIENTES c
LEFT   JOIN PEDIDOS p ON c.id_cliente = p.id_cliente
ORDER  BY c.nome;
-- Gabriela e Henrique aparecem com id_pedido=NULL
```

```sql
-- Clientes que NUNCA fizeram pedido (o uso clássico!)
SELECT c.nome, c.email, c.estado
FROM   CLIENTES c
LEFT   JOIN PEDIDOS p ON c.id_cliente = p.id_cliente
WHERE  p.id_pedido IS NULL  -- só os sem par!
ORDER  BY c.nome;

-- Todos os produtos (com e sem venda)
SELECT pr.nome_produto,
       pr.preco,
       COUNT(ip.id_item) AS vezes_vendido,
       ISNULL(SUM(ip.quantidade),0) AS qtd_total_vendida
FROM   PRODUTOS pr
LEFT   JOIN ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto
GROUP  BY pr.nome_produto, pr.preco
ORDER  BY vezes_vendido DESC;
```

💻 RIGHT JOIN — Incluindo Todos da Tabela Direita

```sql
-- RIGHT JOIN: retorna TODOS os registros da tabela à DIREITA
-- Registros sem par na esquerda aparecem com NULL

-- RIGHT JOIN: todos os pedidos (mesmo sem funcionário associado)
SELECT f.nome     AS vendedor,
       p.id_pedido,
       p.status,
       p.total
FROM   FUNCIONARIOS f
RIGHT  JOIN PEDIDOS p ON f.id_funcionario = p.id_funcionario
ORDER  BY p.id_pedido;

-- IMPORTANTE: RIGHT JOIN pode ser reescrito como LEFT JOIN!
-- Equivalente ao RIGHT JOIN acima (invertendo a ordem das tabelas):
SELECT f.nome     AS vendedor,
       p.id_pedido, p.status, p.total
FROM   PEDIDOS p
LEFT   JOIN FUNCIONARIOS f ON p.id_funcionario = f.id_funcionario
ORDER  BY p.id_pedido;
-- Resultado idêntico! Por isso RIGHT JOIN é menos usado

-- Boa prática: prefira LEFT JOIN (mais legível e consistente)
-- Coloque sempre a tabela 'principal' à ESQUERDA
-- e use LEFT JOIN para incluir os opcionais

-- Pedidos sem funcionário associado (venda direta)
SELECT p.id_pedido, p.status, p.total
FROM   FUNCIONARIOS f
RIGHT  JOIN PEDIDOS p ON f.id_funcionario = p.id_funcionario
WHERE  f.id_funcionario IS NULL;
```

💻 FULL JOIN — Todos de Ambos os Lados

```
-- FULL OUTER JOIN: retorna TODOS de A e TODOS de B
-- Sem par: NULL nos campos da tabela sem correspondência
-- Todos os clientes E todos os pedidos (com ou sem par)
SELECT c.nome          AS cliente,
       c.estado,
       p.id_pedido,
       p.status
FROM   CLIENTES c
FULL   OUTER JOIN PEDIDOS p ON c.id_cliente = p.id_cliente
ORDER  BY c.nome, p.id_pedido;
-- Inclui: clientes sem pedido (NULL no pedido)
--         pedidos sem cliente (NULL no cliente) -- se existir

-- Encontrar inconsistências (NULLs em qualquer lado)
SELECT ISNULL(c.nome,'SEM CLIENTE')    AS cliente,
       ISNULL(CAST(p.id_pedido AS VARCHAR),'SEM PEDIDO') AS pedido,
       CASE WHEN c.id_cliente IS NULL THEN 'Pedido órfão'
            WHEN p.id_pedido   IS NULL THEN 'Cliente sem compra'
            ELSE 'OK'
       END AS situacao
FROM   CLIENTES c
FULL   OUTER JOIN PEDIDOS p ON c.id_cliente = p.id_cliente
WHERE  c.id_cliente IS NULL OR p.id_pedido IS NULL
ORDER  BY situacao, c.nome;
-- FULL JOIN é mais usado em: auditoria de dados,
-- comparação de duas fontes, detecção de inconsistências
```

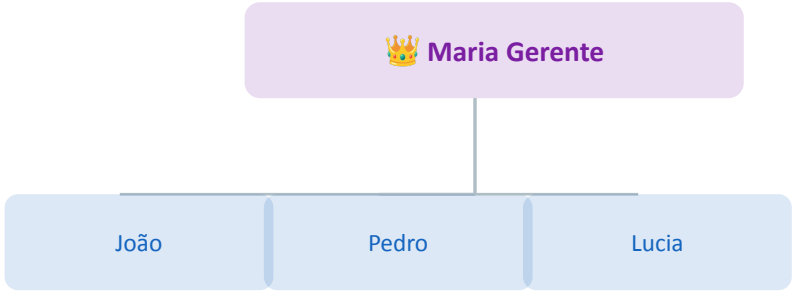
SELF JOIN — Uma Tabela se Une a Si Mesma

SELF JOIN une uma tabela com ela mesma usando aliases diferentes. É usado para hierarquias (funcionário → gerente), comparações entre registros da mesma tabela e estruturas recursivas.

FUNCIONARIOS (com gerente_id)

id	nome	gerente_id
1	Maria Gerente	NULL
2	João Vendedor	1
3	Pedro Estoque	1
4	Lucia Caixa	1

SELF JOIN — Hierarquia Funcionário → Gerente




```
SELECT f.nome AS funcionario,
       g.nome AS gerente
FROM   FUNCIONARIOS f
LEFT   JOIN FUNCIONARIOS g ON f.gerente_id = g.id_funcionario;
```

💻 SELF JOIN — Exemplos Práticos

```
-- SELF JOIN clássico: funcionário e seu gerente
-- Precisamos adicionar coluna gerente_id à tabela FUNCIONARIOS
ALTER TABLE FUNCIONARIOS ADD gerente_id INT NULL;
ALTER TABLE FUNCIONARIOS
ADD CONSTRAINT FK_FUNC_GERENTE
    FOREIGN KEY (gerente_id) REFERENCES FUNCIONARIOS(id_funcionario);
GO
-- Definir hierarquia: João, Pedro e Lucia reportam à Maria
UPDATE FUNCIONARIOS SET gerente_id = 2 WHERE id_funcionario IN (1,3,4);
GO -- (assumindo Maria=2, João=1, Pedro=3, Lucia=4)
-- SELF JOIN: listar funcionário e seu gerente
SELECT f.nome                          AS funcionario,
       f.cargo,
       ISNULL(g.nome, 'É o gerente!')   AS gerente
FROM   FUNCIONARIOS f
LEFT   JOIN FUNCIONARIOS g ON f.gerente_id = g.id_funcionario
ORDER  BY g.nome, f.nome;
-- Comparar preços de produtos da mesma categoria
SELECT p1.nome_produto AS produto,
       p1.preco        AS preco_a,
       p2.nome_produto AS produto_comparado,
       p2.preco        AS preco_b,
       p1.preco - p2.preco AS diferenca
FROM   PRODUTOS p1
JOIN   PRODUTOS p2 ON p1.id_categoria = p2.id_categoria
                  AND p1.id_produto  < p2.id_produto
ORDER  BY p1.id_categoria, ABS(p1.preco-p2.preco) DESC;
```

INNER vs LEFT — Escolhendo o JOIN Certo

❓ Listar pedidos com nome do cliente

✅ INNER JOIN: Todo pedido TEM um cliente. Não há pedidos sem cliente (FK garante).

Desnecessário aqui

❓ Listar TODOS clientes (compraram ou não)

❌ INNER JOIN: Exclui clientes sem pedido — resultado incompleto!

✅ LEFT JOIN: Inclui todos os clientes. Sem pedido: NULL nos campos do pedido.

❓ Relatório de produtos com qtd vendida

❌ INNER JOIN: Produtos nunca vendidos desaparecem do relatório!

✅ LEFT JOIN: Todos os produtos aparecem. Sem venda: COUNT=0, SUM=NULL.

❓ Faturamento por funcionário (só quem vendeu)

✅ INNER JOIN: Se só quer quem vendeu, INNER JOIN está correto.

Depende: LEFT JOIN inclui funcionários sem venda (faturamento=NULL/0).

💻 Combinando Tipos de JOIN — Relatório Completo

```sql
-- Relatório: TODOS os produtos com total vendido
-- (incluindo produtos nunca vendidos com 0)
SELECT pr.nome_produto,
       cat.nome_categoria,
       pr.preco,
       pr.estoque,
       COUNT(ip.id_item)          AS vezes_vendido,
       ISNULL(SUM(ip.quantidade), 0) AS total_unidades,
       ISNULL(SUM(ip.quantidade*ip.preco_unitario), 0) AS faturamento,
       CASE WHEN COUNT(ip.id_item)=0 THEN ' 🔴 Nunca vendido'
            WHEN COUNT(ip.id_item)<3  THEN ' 🟡 Baixo giro'
            ELSE                           ' 🟢 Bom giro'
       END AS situacao
FROM   PRODUTOS    pr
INNER  JOIN CATEGORIAS  cat ON pr.id_categoria = cat.id_categoria
LEFT   JOIN ITENS_PEDIDO ip  ON pr.id_produto  = ip.id_produto
GROUP  BY pr.nome_produto, cat.nome_categoria, pr.preco, pr.estoque
ORDER  BY faturamento DESC;

-- Note: INNER JOIN com CATEGORIAS (todo produto TEM categoria)
--       LEFT JOIN com ITENS_PEDIDO (produto pode nunca ter sido vendido)
-- ISNULL(...,0) converte NULL em 0 para exibição profissional
```

💻 Auditoria de Dados — Encontrando Inconsistências

```sql
-- 1. Clientes sem nenhum pedido (potencial para campanha)
SELECT c.id_cliente, c.nome, c.email, c.estado
FROM   CLIENTES c
LEFT   JOIN PEDIDOS p ON c.id_cliente = p.id_cliente
WHERE  p.id_pedido IS NULL
ORDER  BY c.nome;
-- 2. Produtos nunca vendidos (revisar catálogo)
SELECT pr.id_produto, pr.nome_produto, pr.preco, cat.nome_categoria
FROM   PRODUTOS pr
INNER  JOIN CATEGORIAS   cat ON pr.id_categoria = cat.id_categoria
LEFT   JOIN ITENS_PEDIDO ip  ON pr.id_produto   = ip.id_produto
WHERE  ip.id_item IS NULL
ORDER  BY cat.nome_categoria, pr.nome_produto;
-- 3. Funcionários sem nenhuma venda
SELECT f.id_funcionario, f.nome, f.cargo
FROM   FUNCIONARIOS f
LEFT   JOIN PEDIDOS p ON f.id_funcionario = p.id_funcionario
WHERE  p.id_pedido IS NULL
ORDER  BY f.nome;

-- 4. Pedidos com itens mas sem total calculado
SELECT p.id_pedido, p.status, p.total,
       SUM(ip.quantidade*ip.preco_unitario) AS total_calculado
FROM   PEDIDOS p
LEFT   JOIN ITENS_PEDIDO ip ON p.id_pedido = ip.id_pedido
GROUP  BY p.id_pedido, p.status, p.total
HAVING p.total <> SUM(ip.quantidade*ip.preco_unitario)
    OR p.total IS NULL;
```

✏️ Exercícios Propostos — Aula 11

01 — Básico

Use LEFT JOIN para listar TODOS os clientes com seus pedidos (se existirem). Mostre nome, estado, id_pedido e status. Clientes sem pedido devem aparecer com NULL.

02 — Básico

Use LEFT JOIN + WHERE IS NULL para encontrar: (a) clientes sem pedido, (b) produtos nunca vendidos. Mostre nome e informações relevantes de cada um.

03 — Médio

Crie um relatório de desempenho completo: TODOS os funcionários com total de pedidos (COUNT) e faturamento (SUM). Funcionários sem venda devem aparecer com 0. Use LEFT JOIN + GROUP BY + ISNULL.

04 — Médio

Use FULL OUTER JOIN entre CLIENTES e PEDIDOS. Classifique cada linha como 'Cliente sem compra', 'Pedido normal' ou 'Pedido órfão' (sem cliente). Use CASE + IS NULL.

05 — Avançado

Crie o relatório final de produtos: nome_produto, categoria, preço, total_unidades_vendidas, faturamento, situacao (Nunca vendido/Baixo/Normal/Alto giro). Misture INNER JOIN (categoria) e LEFT JOIN (itens). Ordene por faturamento DESC.

✅ Respostas Comentadas — Exercícios 02 e 03

```sql
-- EX 02a: Clientes SEM pedido (LEFT JOIN + WHERE IS NULL)
SELECT c.nome, c.email, c.estado
FROM   CLIENTES c
LEFT   JOIN PEDIDOS p ON c.id_cliente = p.id_cliente
WHERE  p.id_pedido IS NULL
ORDER  BY c.nome;

-- EX 02b: Produtos NUNCA vendidos
SELECT pr.nome_produto, pr.preco, cat.nome_categoria
FROM   PRODUTOS pr
INNER  JOIN CATEGORIAS cat ON pr.id_categoria=cat.id_categoria
LEFT   JOIN ITENS_PEDIDO ip ON pr.id_produto=ip.id_produto
WHERE  ip.id_item IS NULL
ORDER  BY cat.nome_categoria;

-- EX 03: TODOS funcionários com métricas (zeros incluídos)
SELECT f.nome                                AS vendedor,
       f.cargo,
       COUNT(p.id_pedido)                    AS total_pedidos,
       ISNULL(SUM(p.total), 0)               AS faturamento,
       ISNULL(ROUND(AVG(p.total),2), 0)      AS ticket_medio
FROM   FUNCIONARIOS f
LEFT   JOIN PEDIDOS p ON f.id_funcionario = p.id_funcionario
GROUP  BY f.nome, f.cargo
ORDER  BY faturamento DESC;

-- ISNULL(SUM(...),0): quando não há pedidos, SUM=NULL → 0
-- COUNT(p.id_pedido): conta NULL como 0 automaticamente!
```

🏆 Desafio da Aula 11

📋 Contexto: O setor de qualidade de dados e o marketing precisam de relatórios de auditoria e segmentação completos — incluindo registros sem correspondência (órfãos).

🎯 Relatórios e auditorias:

A1 Painel de clientes: TODOS com total de pedidos (COUNT), gasto total (SUM), último pedido (MAX data) e situação ('VIP'>R$1000, 'Regular', 'Inativo' sem pedido).

A2 Catálogo de produtos: TODOS com categoria, qtd vendida, faturamento, situação de estoque e giro (Nunca vendido/Baixo/Normal). Misture INNER + LEFT JOIN.

A3 Relatório de vendedores: TODOS os funcionários com pedidos entregues/cancelados (COUNT CASE), faturamento real e comissão 5%. Zeros para quem não vendeu.

A4 Auditoria FULL JOIN: detectar pedidos sem cliente válido e clientes sem pedido. Classificar e contar cada tipo de inconsistência encontrada.

A5 BÔNUS: Análise de cohort simples — para cada mês de cadastro dos clientes (YEAR+MONTH de data_cadastro), quantos fizeram pedido e qual o gasto médio. LEFT JOIN + GROUP BY.

📚 Resumo — Aula 11: JOINs Avançados

⬅️ LEFT JOIN

TODOS da esquerda + correspondência ou NULL. O mais usado de todos

➡️ RIGHT JOIN

TODOS da direita + correspondência ou NULL. Pode ser reescrito como LEFT JOIN

↔️ FULL JOIN

TODOS de ambos os lados. Ideal para auditoria e detecção de inconsistências

🔄 SELF JOIN

Tabela com ela mesma. Alias obrigatórios. Hierarquias e comparações

🔍 LEFT + WHERE IS NULL

Padrão para encontrar registros 'órfãos' — sem correspondência no outro lado

🆚 INNER vs LEFT

FK garante par? INNER. Pode não ter par? LEFT. Pergunta-chave para decidir

0️⃣ ISNULL para zeros

ISNULL(SUM(...),0) e COUNT(coluna)=0: relatórios com zeros no lugar de NULL

🔀 Misturar JOINs

INNER para relações garantidas por FK, LEFT para relações opcionais na mesma query

🚀 Próxima Aula 12: Operações entre Conjuntos

Aula 12 — O que vem por aí:

∪ **UNION / UNION ALL**	Combinar resultados de múltiplas queries em uma lista única
∩ **INTERSECT**	O que há em comum entre dois conjuntos de resultados
∖ **EXCEPT**	O que está em A mas não em B — diferença entre conjuntos
📊 **Casos com dados reais**	Aplicando operações de conjunto no SISTEMA_VENDAS
🆚 **JOIN vs UNION**	Quando cada abordagem é a mais adequada
⚡ **Performance**	UNION ALL vs UNION — quando e por que usar cada um

📝 **Para a próxima aula: conclua os relatórios de auditoria e pense: UNION vs JOIN — qual a diferença fundamental?**